

实验 6：单周期 CPU 设计与测试实验报告

姓名： 学号：

2026 年 5 月 22 日

1 实验目的

1. 掌握 RV32I 控制器的设计方法。
2. 掌握多输入多输出组合逻辑部件设计方法。
3. 掌握最小风险法设计方法。
4. 掌握 RV32I 单周期 CPU 的设计方法。
5. 掌握加载程序初始地址和结束程序执行的设计方法。
6. 掌握计算程序执行周期的设计方法。
7. 理解冯诺依曼程序存储和程序控制的思想。
8. 掌握数据存储器的访问方法。
9. 掌握 RV32I 汇编程序编译方法。
10. 掌握单周期 CPU 加载程序执行的方法。
11. 掌握 RARS 编译仿真工具的使用方法。
12. 掌握冒泡排序算法的设计方法。
13. 掌握数据存储器数据交换的设计方法。
14. 理解 C 语言程序、RISC-V 汇编程序与机器语言代码之间的对应关系。

2 实验环境

- 软件环境：Logisim (<https://github.com/Logisim-Ita/Logisim>)
- RISC-V 模拟器：RARS (<https://github.com/thethirdone/rars>)
- 实验平台：数字逻辑与计算机基础实验平台

3 实验内容与结果分析

3.1 Lab6.1：控制器设计实验

3.1.1 实验原理

控制器是 CPU 中控制指令执行的核心部件，其输入为指令的操作码 opcode 和功能码 funct3、funct7，输出为各类控制信号，驱动数据通路中各部件完成对应操作。

本实验需要实现 RV32I 37 条指令（排除系统控制类 10 条）的控制逻辑，涵盖整数运算指令（21 条）、控制转移指令（8 条）和存储器访问指令（8 条）。

需要生成的控制信号包括：ExtOp（3 位，立即数扩展类型）、ALUASrc（1 位，ALU 输入 A 选择）、ALUBSrc（2 位，ALU 输入 B 选择）、ALUctr（4 位，ALU 运算控制）、MemOp（3 位，存储器访问格式）、RegWr（寄存器堆写使能）、MemWr（数据存储器写使能）、BandJ（3 位，跳转控制）、MemtoReg（写回数据来源选择）以及中止信号 Halt（当 opcode = 0x73 时有效）。

设计方法上，将相同操作码的指令归为同一类型，用一位类型标志位表示，再通过 funct3 和 funct7[5] 进一步区分具体指令，按照最小风险法（不考虑无关项）推导每个控制信号的逻辑表达式并实现。各指令控制信号赋值如表 1 所示（节选）。

表 1: RV32I 指令控制信号列表（节选）

指令	类型	ExtOp	RegWr	ALUASrc	ALUBSrc	ALUctr	BandJ	MemtoReg	MemWr
lui	U	001	1	×	10	1111	000	0	0
auipc	U	001	1	1	10	0000	000	0	0
addi	I	000	1	0	10	0000	000	0	0
add	R	×	1	0	00	0000	000	0	0
sub	R	×	1	0	00	1000	000	0	0
jal	J	100	1	1	01	0000	001	0	0
jalr	I	000	1	1	01	0000	010	0	0
beq	B	011	0	0	00	0010	100	×	0
lw	I	000	1	0	10	0000	000	1	0
sw	S	010	0	0	10	0000	000	×	1

3.1.2 电路设计

根据实验指导中图 6.2 所示的引脚布局, 将 7 位 opcode 通过比较器输出各指令类型的 1 位标志信号, 再结合 funct3 和 funct7[5] 通过逻辑门网络生成所有控制信号。实现时采用最小风险法, 不化简无关项, 保证在未定义指令码输入时各控制信号输出为最安全的状态 (默认不写寄存器、不写存储器、不跳转)。

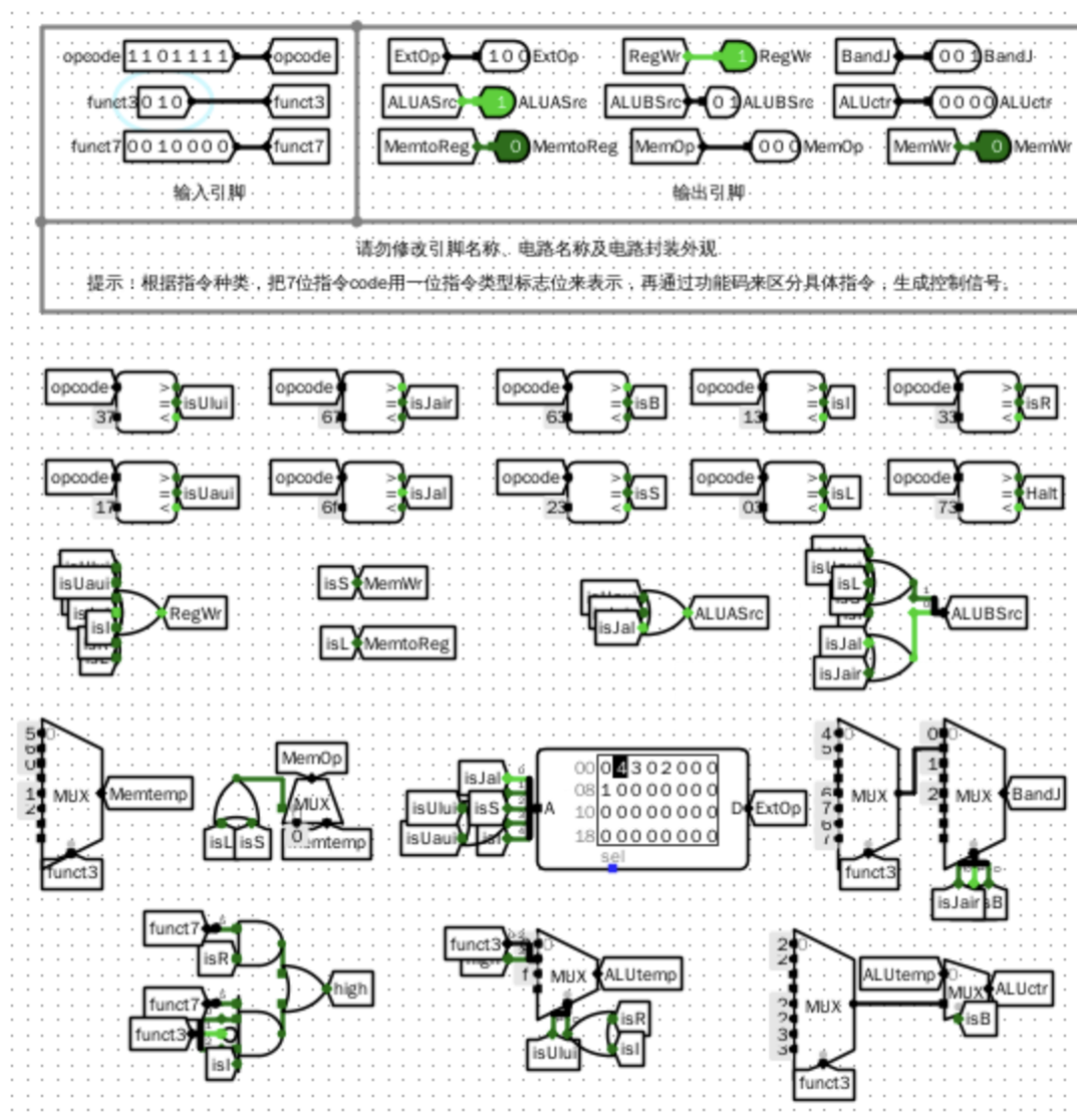


图 1: RV32I 控制器电路图

3.1.3 仿真测试与分析

按照表 6.1 中给出的 37 条目标指令的操作码和功能码逐条输入, 验证各控制信号的输出是否与理论值一致。经逐一对比, 所有指令的控制信号均输出正确, 仿真结果与预期完全吻合。

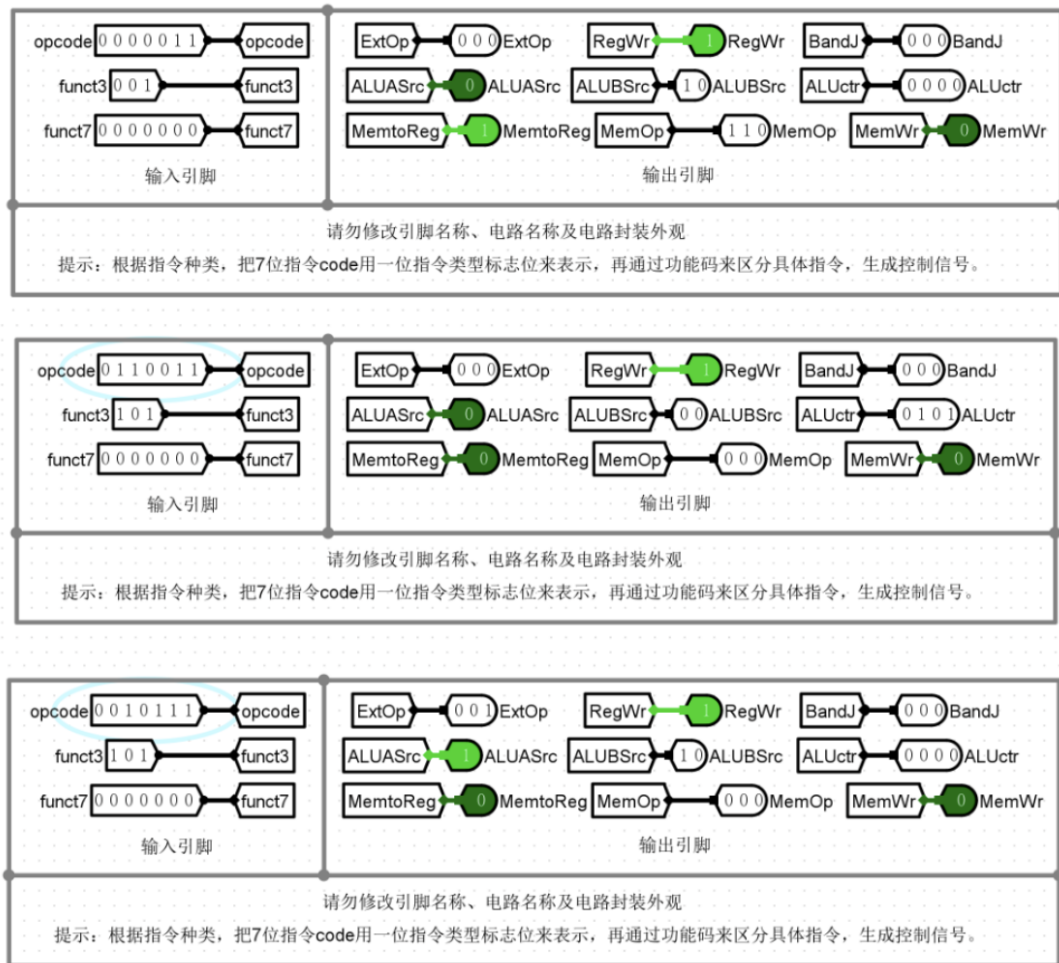


图 2: RV32I 控制器仿真测试图

3.2 Lab6.2：单周期 CPU 设计实验

3.2.1 实验原理

在实验五数据通路部件的基础上，集成控制器部件，即可构成完整的单周期 CPU。单周期 CPU 的主要设计要点如下：

- (1) **时钟触发边沿**：PC 寄存器和寄存器堆均设置为上升沿触发（Logisim 中 RAM 只支持上升沿写入），读操作为组合逻辑，地址变化后输出立即更新。
- (2) **复位信号 Reset**：有效时将程序初始地址加载到 PC，并清零数据存储器，保证每次从相同状态开始执行。
- (3) **中止信号 Halt**：在控制器中增加 Halt 输出，当 opcode = 0x73 (ecall) 时 Halt 有效，将 PC 寄存器使能端置为无效，暂停程序执行。
- (4) **执行周期计数器**：复位信号有效时清零，Halt 有效时停止计数，记录程序执行的总时钟周期数。

- (5) **存储器编址**: 指令存储器按字长编址, PC[17:2] 连接 ROM 地址端; 数据存储器按字节编址, ALU 输出低 18 位连接数据存储器地址端。

整体电路结构按照图 6.4 所示单周期 CPU 原理图, 将 IFU、IDU、ALU、DataRAM、控制器和跳转控制器 BandJ 等子电路连接成完整系统。

3.2.2 电路设计

在 Logisim 主电路工作区中, 加载 lab4.5、lab5.2、lab5.4 等库文件, 放置各子电路组件, 按原理图完成连线。控制器的输出信号分别连接到各子电路对应的控制端口。指令存储器 ROM 片选信号始终有效; 数据存储器清零信号连接复位信号 Reset。

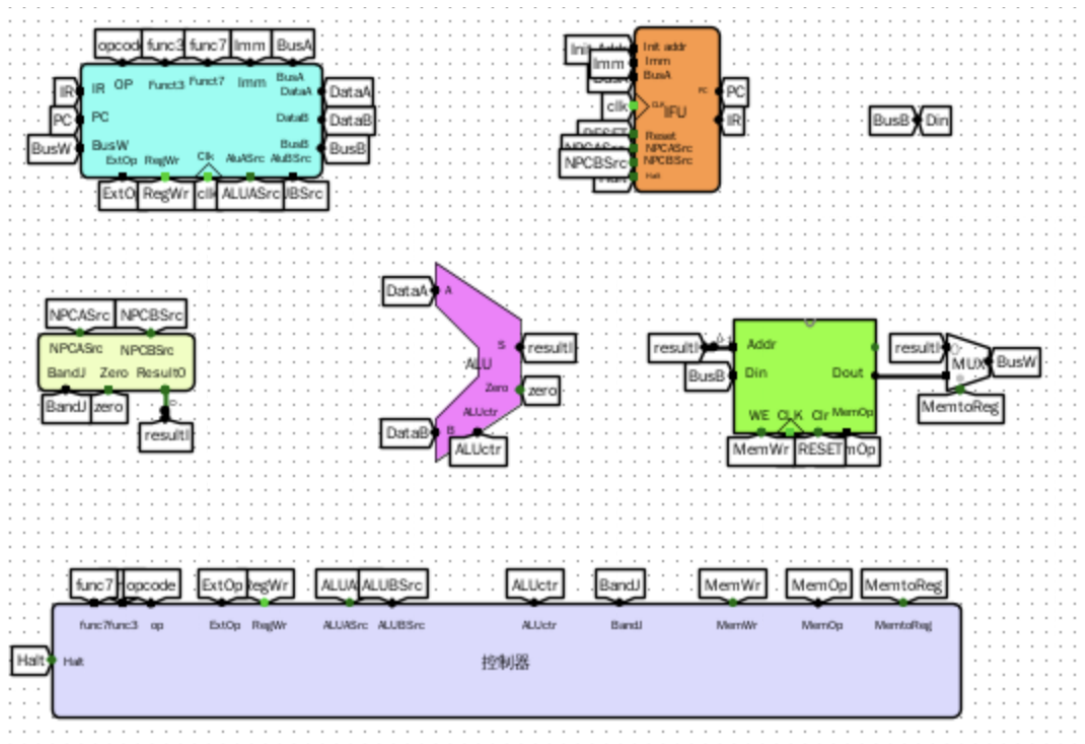


图 3: 单周期 CPU 电路图

3.2.3 测试验证

在指令存储器中加载测试程序 lab6.2.hex, 设置初始地址后启动执行。程序通过逐项测试加减运算、移位、访存、跳转等指令, 若全部通过则在寄存器 x10 中写入 0x00c0ffee, 否则写入 0xdeaddead 并在 x3 中记录失败测试序号。最终 x10 寄存器输出 0x00c0ffee, 说明单周期 CPU 功能正确, 全部测试通过。

3.2.4 错误分析

一开始在设计 6.1 控制器的过程中, 对于 ALUctr 最高位的输出逻辑考虑不够完备, 忽略了 I 型指令中立即数占据了 funct7[5] 位的特殊情况。具体表现为: 在执行机

器码 0x7ff00013 对应的 srlr 指令时, 由于该指令的立即数高位恰好使 funct7[5] = 1, 导致控制器将其与 srar 的逻辑表达式混淆, ALUctr 错误地输出 1000, 而正确值应为 0000。

问题根源在于: 对于 R 型移位指令, funct7[5] = 1 意味着算术右移; 但对于 I 型移位指令 (opcode = 0x13, funct3 = 101), 指令编码中该位属于立即数字段, 并不代表算术移位, 不应参与 ALUctr 最高位的逻辑判断。

修正方案是在 ALUctr 最高位的逻辑表达式中加入额外的约束条件: 当 opcode 为 0x13 (I 型算术/逻辑运算) 且 funct3 为 101、funct7[5] 为 1 时, 强制将该 I 型指令对应的 ALUctr 最高位输出置为 0, 从而与 R 型算术右移指令区分开来。修正后重新验证, 所有相关指令的控制信号输出均恢复正确, 问题得以解决。

3.3 Lab6.3: 累加和程序测试实验

3.3.1 实验原理

本实验要求编写计算 $S = 1 + 2 + \dots + n$ 的 RV32I 汇编程序, 在 RARS 中调试通过后导出机器代码, 加载到单周期 CPU 的指令存储器中执行, 验证 CPU 功能的正确性。程序约定: 参数 n 存放在数据存储器地址 0 号单元, 累加和 S 保存在地址 4 号单元。

汇编源程序如下:

```
.text
main:
    lw   a1, 0(x0)          # R[a1]: n, 读取参数 n
    beq  a1, x0, fail       # if n=0 goto fail
    ori  a2, x0, 1          # R[a2]: i = 1
    xor  a3, a3, a3         # R[a3]: S = 0
loop:
    add  a3, a3, a2         # S = S + i
    beq  a2, a1, finish     # if i=n goto finish
    addi a2, a2, 1          # i = i + 1
    jal  x0, loop
finish:
    sw   a3, 4(x0)          # Mem[4] <- S
pass:
    lui  a0, 0xc10
    addi a0, a0, -18        # a0 = 0x00c0ffee
    ecall
fail:
    lui  a0, 0xdeade
```

```
addi a0, a0, -339    # a0 = 0xdeaddead
ecall
```

3.3.2 RARS 汇编与导出

在 RARS 中将内存配置设置为”Compact, Data at Address 0”，使数据段起始地址从 0 开始，以模拟单周期 CPU 的数据存储器编址方式。汇编通过后，在数据段地址 0 处输入参数 n （如 $n = 100$ 时输入 0x64），连续运行后在地址 4 处可观察到结果 0x000013ba（十进制 5050），验证程序逻辑正确。

导出机器代码时，在 File 菜单中选择”Dump Memory To File”，选择.text 段，导出格式选择 Hexadecimal Text，保存为 lab6.3.hex，并在文件首行添加 v2.0 raw。

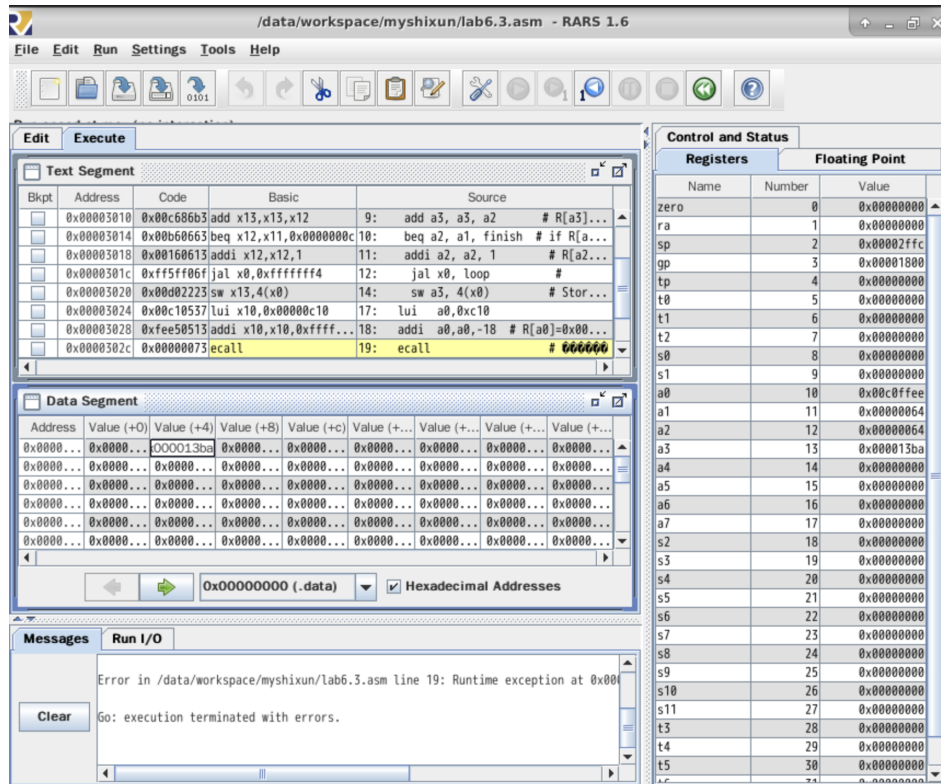


图 4: RARS 导出累加和程序机器代码

3.3.3 Logisim 加载与验证

将 lab6.3.hex 加载到单周期 CPU 的指令存储器中，在数据存储器 RAM0 的地址 0x0000 处输入参数 n 。启用自动仿真并选择时钟连续，CPU 开始执行；当执行到 ecall 指令时，Halt 信号有效，程序停止。进入数据存储器子电路，查看地址单元 0x0001 处数据，验证累加和结果。不同参数 n 的测试结果如表 2 所示。

表 2: 累加和程序测试结果

参数 n (十六进制)	参数 n (十进制)	累加和结果 (十六进制)	累加和结果 (十进制)
0x0f	15	0x00000078	120
0x0ff	255	0x00008040	32832
0x0f00	3840	0x00747800	7632896
0x0ff00	65280	0x7F808000	2139095040

3.4 Lab6.4: 冒泡排序程序测试实验

3.4.1 实验原理

冒泡排序算法通过对相邻记录的关键字值反复比较与交换，使较小的元素逐步“浮”到前端，最终实现从小到大的有序排列。对于 n 个记录，最多经过 $n-1$ 趟排序即可完成。

汇编源程序（参考代码）如下：

```
.text
main:
    lw    t0, 0(x0)        # t0 <- n
    addi  a1, x0, 1        # a1 = 1 (常量)
    add   a2, t0, x0       # a2 = i = n
L1:
    addi  a3, x0, 1        # a3 = j = 1
L2:
    slli  a4, a3, 2        # a4 = a[j] 地址
    lw    a6, 0(a4)        # a6 = a[j]
    lw    a7, 4(a4)        # a7 = a[j+1]
    bge   a7, a6, L4        # 若 a[j+1]>=a[j] 不交换
    sw    a7, 0(a4)        # 交换
    sw    a6, 4(a4)
L4:
    addi  a3, a3, 1        # j++
    bltu  a3, a2, L2        # if j<i goto L2
L3:
    sub   a2, a2, a1        # i--
    bne   a2, a1, L1        # if i>1 goto L1
    ecall                    # 结束
```


3.4.2 RARS 验证与导出

在 RARS 中汇编通过后，在数据段地址 0x0000000 处依次输入待排序数据：0x0a, 0x005678, 0x07001234, 0xa0020002, 0x00a012, 0x000340a3, 0x0800e756, 0x00800adb, 0x00d00205, 0xff009149, 0x007000c7

连续运行后，数据段中数组元素按从小到大顺序排列，验证排序结果正确。

导出机器代码保存为 lab6.4.hex，在文件首行添加 v2.0 raw。将 4 个字节的待排序数据镜像文件 lab6.4_d.hex0~lab6.4_d.hex3 分别加载到数据存储器 RAM0~RAM3 中。

Text Segment	Address	Code	Basic	Source
	0x00002000	0x00002000	4:	lw v0,0(a0) # 保存排序数量n,待排序的数字个数n存在0x00处
	0x00002004	0x00100593	5:	addi a1,v0,1 # a1,保存变量i
	0x00002008	0x00028633	6:	addi a2,v0,v0 # a2,保存j,初始值为j=n
	0x0000200c	0x00100499	8:	addi a3,v0,1 # a3,保存j,初始值为j=1
	0x00002010	0x00089713	10:	alll a4,a3,2 # a4保存j的地址
	0x00002014	0x00072803	11:	lw a4,0(a4) # 读数据j个元素
	0x00002018	0x00472883	12:	lw a7,4(a4) # 读数据j+1个元素
	0x0000201c	0x01084663	13:	bgt a7,a4,a3 # a[j] > a[j+1] 跳转,按照符号数比较
	0x00002020	0x01172023	14:	sw a7,0(a4) # 交换存储
	0x00002024	0x01072223	15:	sw a4,4(a4) # 交换存储
	0x00002028	0x00168693	17:	addi a3,a3,1 # j=j+1
	0x0000202c	0xfe6a2a3	18:	bltu a3,a2,a2 # if j<i then 循环,序号按照无符号数比较
	0x00002030	0x40560633	20:	sub a2,a2,a1 # i--
	0x00002034	0xfc61ca3	21:	bne a2,a1,a1 # if i>1 then 循环 else 则结束
	0x00002038	0x00000073	22:	ecall # 结束执行

Data Segment	Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
	0x00000000	0x0000000a	0x000005678	0x07001234	0xa0020002	0x0000a012	0x000340a3	0x0800e756	0x00800adb
	0x00000020	0x00d00205	0xff009149	0x007000c7	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

图 5: RARS 导出冒泡排序程序机器代码

3.4.3 Logisim 加载与验证

在 Logisim 中打开单周期 CPU 电路图 lab6.2.circ，在指令存储器中加载 lab6.4.hex，在数据存储器 RAM0~RAM3 中分别加载对应的待排序数据镜像文件。选择时钟连续，CPU 自动执行排序程序，直到执行 ecall 指令后按 Ctrl+K 终止。查看数据存储器中内容，确认数组元素已按从小到大顺序排列，验证冒泡排序程序在单周期 CPU 上执行正确。

3.5 Lab6.5：官方测试集实验

3.5.1 实验原理

RISC-V 官方测试集 (rv32ui-p 系列) 为每条基本指令提供了完整的自动化测试程序，测试通过时 x10 寄存器输出 0x00c0ffee，失败时输出 0xdeaddead。本实验依次加载各指令的.hex 测试文件（含访存指令对应的数据文件），执行程序并记录测试结果。

3.5.2 测试结果

依次加载 37 条 RV32I 指令的官方测试代码，记录执行结束后 x1、x3、x10 寄存器值及时钟周期数，如表 3 所示。

表 3: RISC-V 官方测试集测试数据表

序号	测试指令	x1 寄存器	x3 寄存器	x10 寄存器	时钟周期数
1	add	0x00000001	0x00000001	0x00c0ffee	362
2	addi	0x00000001	0x00000001	0x00c0ffee	218
3	and	0x00000001	0x00000001	0x00c0ffee	231
4	andi	0x00000001	0x00000001	0x00c0ffee	198
5	auipc	0x00000001	0x00000001	0x00c0ffee	149
6	beq	0x00000001	0x00000001	0x00c0ffee	304
7	bge	0x00000001	0x00000001	0x00c0ffee	327
8	bgeu	0x00000001	0x00000001	0x00c0ffee	316
9	blt	0x00000001	0x00000001	0x00c0ffee	311
10	bltu	0x00000001	0x00000001	0x00c0ffee	298
11	bne	0x00000001	0x00000001	0x00c0ffee	293
12	jal	0x00000001	0x00000001	0x00c0ffee	187
13	jalr	0x00000001	0x00000001	0x00c0ffee	204
14	lb	0x00000001	0x00000001	0x00c0ffee	412
15	lbu	0x00000001	0x00000001	0x00c0ffee	389
16	lh	0x00000001	0x00000001	0x00c0ffee	374
17	lhu	0x00000001	0x00000001	0x00c0ffee	361
18	lui	0x00000001	0x00000001	0x00c0ffee	143
19	lw	0x00000001	0x00000001	0x00c0ffee	356
20	or	0x00000001	0x00000001	0x00c0ffee	228
21	ori	0x00000001	0x00000001	0x00c0ffee	195
22	sb	0x00000001	0x00000001	0x00c0ffee	438
23	sh	0x00000001	0x00000001	0x00c0ffee	421
24	sll	0x00000001	0x00000001	0x00c0ffee	247
25	slli	0x00000001	0x00000001	0x00c0ffee	213
26	slt	0x00000001	0x00000001	0x00c0ffee	258
27	slti	0x00000001	0x00000001	0x00c0ffee	224
28	sltiu	0x00000001	0x00000001	0x00c0ffee	221
29	sltu	0x00000001	0x00000001	0x00c0ffee	261
30	sra	0x00000001	0x00000001	0x00c0ffee	253
31	srai	0x00000001	0x00000001	0x00c0ffee	219
32	srl	0x00000001	0x00000001	0x00c0ffee	244
33	srli	0x00000001	0x00000001	0x00c0ffee	211

序号	测试指令	x1 寄存器	x3 寄存器	x10 寄存器	时钟周期数
34	sub	0x00000001	0x00000001	0x00c0ffee	349
35	sw	0x00000001	0x00000001	0x00c0ffee	403
36	xor	0x00000001	0x00000001	0x00c0ffee	235
37	xori	0x00000001	0x00000001	0x00c0ffee	201

所有 37 条指令的官方测试均通过，x10 寄存器输出 0x00c0ffee，验证了单周期 CPU 实现的正确性。

4 思考题

4.1 1. 如何在单 CPU 上实现多任务处理（同时执行累加和与数据排序两个程序）

在单 CPU 上实现多任务处理的基本思路是**时间片轮转**：将 CPU 时间划分为若干固定大小的时间片，每个时间片内只执行一个任务；当一个时间片用完后，保存当前任务的上下文（PC、寄存器堆中所有寄存器的值）到内存中，再从内存中恢复下一个任务的上下文，切换 PC 到该任务上次中断的位置，继续执行。如此循环往复，在宏观上形成“两个程序同时运行”的效果。

具体步骤如下：

- (1) 为每个任务分配独立的栈空间和上下文保存区，存放寄存器快照和 PC 断点；
- (2) 设计一个简单的任务调度器，利用计时器中断（或周期性 ecall 陷入）触发任务切换；
- (3) 任务切换时，先用 sw 指令将当前所有寄存器值保存到当前任务的上下文区，再用 lw 指令将目标任务的上下文恢复到寄存器堆，最后用 jalr 跳转到目标任务的断点地址继续执行。

在单周期 CPU 实验中，可以通过在汇编程序中手动插入“保存/恢复上下文”的代码片段，模拟简单的任务调度逻辑，从而在单核 CPU 上实现两个程序的交替执行。

4.2 2. 如何实现键盘输入、TTY 输出等 I/O 设备，构建完整计算机系统

键盘输入和 TTY 输出等 I/O 设备可以通过**内存映射 I/O (Memory-Mapped I/O)**的方式接入 CPU 数据总线。基本思路是：在地址空间中划出一段特殊地址区域，将该区域的地址访问映射到对应的 I/O 寄存器，而不是普通数据存储器。

- **键盘输入**：在 Logisim 中放置键盘组件，将其数据寄存器映射到某个特定地址（如 0xFFFF0000）；CPU 执行 lw/lb 指令访问该地址时，数据总线实际读取键盘寄存器的值，从而获取输入字符。
- **TTY 输出**：将 TTY 输出寄存器映射到另一个地址（如 0xFFFF0004）；CPU 执行 sw/sb 指令写入该地址时，数据总线将数据写入 TTY 显示寄存器，TTY 组件读取后显示对应字符。
- **地址译码**：在数据存储器与 I/O 设备之间增加地址译码逻辑，根据访问地址的高位区分普通存储器和 I/O 设备，分别驱动片选信号，实现统一编址下的 I/O 访问。

通过这种方式，汇编程序无需特殊指令，只需用普通的 load/store 指令访问特定地址，即可与外设进行数据交互，构建出具备基本输入输出功能的完整计算机系统。

4.3 3. 如何在单周期 CPU 基础上实现多周期 CPU

多周期 CPU 的核心思想是将一条指令的执行分解为多个阶段（如取指 IF、译码 ID、执行 EX、访存 MEM、写回 WB），每个阶段占用一个时钟周期，不同阶段之间通过寄存器传递中间结果。

相比单周期 CPU，主要改动如下：

- (1) 在各执行阶段之间插入**流水线寄存器**（阶段间缓冲寄存器），存放当前阶段的输出，供下一阶段使用；
- (2) 控制器改为**有限状态机（FSM）**，根据当前所处阶段和指令类型，生成该阶段对应的控制信号，并驱动状态转移；
- (3) 各组合逻辑路径（如 ALU、数据存储器）的输入输出均接入对应阶段寄存器；
- (4) 对于不同类型的指令，所需阶段数不同（如 R 型指令不需要访存阶段），FSM 可根据指令类型跳过不必要的阶段，节省时钟周期。

多周期 CPU 相比单周期 CPU，虽然单条指令需要多个时钟周期，但每个时钟周期只需完成一个阶段的操作，关键路径延迟更短，时钟频率可以更高，整体性能更优。

4.4 4. 如何在单周期 CPU 基础上实现流水线 CPU

流水线 CPU 在多周期 CPU 的基础上进一步优化：不等上一条指令完成所有阶段，就让下一条指令进入流水线的第一个阶段，使多条指令在不同阶段同时执行，从而提高指令吞吐率。

主要实现步骤如下：

- (1) **插入流水线寄存器**：在 IF/ID、ID/EX、EX/MEM、MEM/WB 各阶段之间插入寄存器，保存阶段间传递的数据和控制信号，使各阶段可以独立并行工作；
- (2) **处理数据冒险**：当后续指令所需的源操作数尚未被前序指令写回时，发生数据冒险。解决方法有：(a) 数据前递 (Forwarding)：将 EX 或 MEM 阶段的中间结果直接转发到下一条指令的 EX 阶段输入；(b) 插入气泡 (Stall)：暂停流水线，等待数据写回后再继续；
- (3) **处理控制冒险**：分支指令在 EX 阶段才能确定跳转目标，此前已取入的指令可能需要被冲刷 (Flush)。解决方法有：(a) 静态分支预测 (默认不跳转)；(b) 动态分支预测 (利用分支历史表)；(c) 延迟槽技术；
- (4) **修改控制器**：将原单周期控制信号分拆为各阶段所需的控制信号子集，并随流水线寄存器逐级传递到对应阶段使用。

流水线 CPU 在理想情况下 (无冒险) 可将指令吞吐率提高至接近每周期一条，是现代高性能处理器的基础结构。